

SIH26168 — Team Explainer

AI/ML-Based Intelligent Dead Reckoning System — What We Built, Why, and How It Works

1. What Are We Actually Building?

This is the single most important thing for everyone on the team to be able to say in one sentence:

The app tells you where you are, continuously, even after GPS is lost. It does NOT tell you which way to turn to reach a destination.

These are two different problems:

- Positioning (what we built): "You are here, and here's how confident we are in that estimate." This is the map, the dot, the growing uncertainty circle.
- Routing / turn-by-turn (NOT what we built): "Turn left in 200m." Google Maps already does this well — it just breaks the moment GPS disappears, because its position estimate goes bad.

Our app solves the upstream problem that routing apps depend on: keeping a reliable position estimate alive through a GNSS blackout (tunnels, underground parking, dense urban canyons), so that whatever app is doing the routing doesn't lose the thread.

If a judge asks "does your app give directions?" — the honest, correct answer is no. It keeps you located; routing is a separate, already-solved problem we deliberately did not rebuild.

2. Why Is This Problem Genuinely Hard?

This is an active academic research area — the original problem statement cited papers from 2021–2025 still working on exactly this. That's why ISRO posed it as a hackathon challenge rather than expecting a finished commercial product. The difficulty is real; the fix was to keep the SYSTEM simple even though the underlying MATH (sensor fusion under uncertainty) is hard.

Every one of the following was a deliberate scope decision, made to keep the project buildable in the time available:

- Single smartphone only — no dedicated hardware, no vehicle CAN-bus access.
- One trained model, one deployment domain — vehicle-only (matches the only real, audited accuracy figure we have).
- No backend, no multi-user system, no admin dashboard.
- No pedestrian/walking mode (descope after the trained pedestrian model was found to be unused and unwired).
- One city's offline map (Coimbatore) instead of statewide coverage (a full Tamil Nadu map was investigated and found to cost 3.5GB+ with real engineering risk, for no real demo benefit).

None of these are things we "couldn't" do — they're things we chose not to do, so the parts we DID build would actually work and be defensible under questioning.

3. Why No Backend?

Because the entire value of the system depends on working with ZERO connectivity — that is the whole point of a GNSS-denied environment. If position tracking required phoning a server, it would break exactly when it's needed most (inside a tunnel with no signal at all).

So all sensor fusion, ML inference, and map matching run entirely on-device, by design — not as a shortcut, but because a backend would be actively wrong for this specific problem.

4. Why No Admin Dashboard / React Web App?

An admin dashboard implies a backend that aggregates multiple users' positions across a fleet — that's a genuinely different product (a logistics/fleet-management platform), not what this problem statement asks for.

It's not a bad idea — fleet tracking is listed as a real future deployment path (ride-hailing, delivery, freight) — but building it now would mean building an entire second system, with its own backend, under deadline pressure, for a demo that doesn't need it. This is a strong bullet point for the "Future Work / Impact" slide, not something to build before the deadline.

5. System Architecture — How It Actually Works

Tech Stack

- Platform: Native Android (Kotlin, Jetpack Compose) — fully on-device.
- Sensor Fusion: Extended Kalman Filter (EKF, 6-state) + Zero-Velocity Update (ZUPT) + Non-Holonomic Constraint (NHC, vehicle motion) + magnetometer-fused heading with live confidence monitoring.
- Machine Learning: 2-layer GRU neural network, deployed on-device via ONNX Runtime. Trained on IO-VNBD — the real vehicle inertial-odometry dataset officially linked on the SIH portal for this exact problem statement.
- Mapping: Offline OpenStreetMap tiles + road network, with map-matching to snap raw position onto the most plausible real road — works with zero connectivity.
- Data: IO-VNBD (primary) + OxIOD (supplementary) — both real, published, peer-reviewed inertial-odometry research datasets.

System Flow

1. Sensors (accelerometer, gyroscope, magnetometer, GPS) feed continuously into the sensor fusion layer.
2. The EKF (with ZUPT and NHC) maintains a running position + velocity estimate.
3. While GPS is available, it directly corrects the estimate.
4. When GPS is lost (blackout), the on-device GRU model predicts short-term displacement from recent motion data.
5. A kinematic gate checks every ML prediction against a physically plausible envelope before accepting it — this stops a bad AI guess from corrupting the position.
6. The fused estimate, plus a growing confidence radius, is map-matched onto the nearest real road and shown live.
7. When GPS returns, the estimate is smoothly reconciled with the fresh fix rather than jumping discontinuously.

6. UI Walkthrough — What Every Number Means

The current screen shows a lot at once. Here is what each part actually means, in plain language:

UI Element	What It Means
GNSS / DEAD RECKONING badge	Which mode is currently active — using real GPS, or running on phone sensors alone because GPS is gone.
GNSS BLACKOUT + Duration	How long the app has been running without GPS.
DR Distance	How far the app thinks you've traveled since GPS was lost, based on sensors alone.

Max Error	An evaluation-only number (compares against real GPS still running silently in the background for testing). Would not exist in a real tunnel — it's how WE check accuracy, not something a real user sees.
ML Latency	How fast the AI model runs, in milliseconds. Proves it is genuinely running on-device, not simulated.
Confidence Radius	"You are probably within this many meters." This is the honest, real-world-usable uncertainty measure — unlike Max Error, it works with no secret GPS access.
Heading Conf.	How much the app currently trusts its own sense of direction (LOW / MEDIUM / HIGH). A wrong sense of direction throws off everything downstream.
GNSS (status row)	Is real GPS currently available.
Navigation (status row)	Which system is actively driving the position estimate right now — GNSS or DR.
Motion	Is the phone detected as stationary, moving, or rotating.
ML Gate (Accepted/Clamped/Rejected)	How many of the AI model's suggested corrections were accepted vs. rejected as physically implausible. This is the safety check protecting the position estimate from bad AI guesses.
ML / EKF (status)	Confirms the AI model and the core math engine are both actively running.
MAP / OFFLINE MAP	Confirms the map works with zero internet connectivity.
Position (lat/long)	Raw GPS-style coordinates of the current estimate.
Speed / Heading / Distance	Current motion readout.
DR Error / Drift	Accuracy statistics for the current session.
Sensor Status	Confirms the phone's accelerometer, gyroscope, and magnetometer are actively supplying data.

A simplified two-view redesign is in progress: a plain "Navigating without GPS" screen with just a map and a High/Medium/Low confidence label for everyday users, with all of the above moved behind a "Show technical details" toggle for judges/engineers who want the full picture.

7. Known Accuracy & Honest Limitations

This is the most important section to get right if asked directly — being honest about limitations is a strength in this project's story, not a weakness, and matches how every audit along the way has been handled.

- Real, audited result: 11.28m mean error, a 54.45% reduction vs. an uncorrected baseline, measured on 27,964 genuinely held-out test windows from IO-VNBD.
- The critical caveat: this figure was measured WITH accurate heading (from the vehicle's own CAN-bus/dual-antenna GPS during testing). Our actual phone-only app has no CAN-bus — it relies on the phone's own magnetometer-fused heading, whose reliability specifically inside a vehicle or a tunnel is not fully characterized.
- Correct framing to use: present 11.28m as "ML displacement-model accuracy, evaluated with reference heading," and separately, explicitly disclose that end-to-end on-device accuracy depends on phone heading quality, which has not yet been isolated or measured. This is not a new admission — it's a limitation the source research itself already discloses.
- Current deployment scope: vehicle-only. Pedestrian/walking use is explicitly future work, not implemented.

8. Frequently Asked Questions

"Isn't this already solved by Google Maps?"

Not for this configuration. Mature automotive dead-reckoning solutions exist, but they depend on dedicated hardware and vehicle CAN-bus access. This problem specifically asks for a bare-smartphone solution, which remains an active, unsolved research area.

"Why not just use GPS or NavIC directly?"

No satellite system can penetrate tunnels, underground structures, or dense high-rises — that's a physical limitation, not an engineering gap satellites can close.

"How accurate is it, really?"

See Section 7 above — report the number with the heading caveat, every time, not just once in fine print.

"Why did we cut [backend / pedestrian mode / statewide map]?"

Each was a deliberate scope decision to keep the shipped system reliable and defensible under a hard deadline, not a sign of giving up on the idea — each is a legitimate, documented "future work" item.

"What happens when GPS comes back?"

The system reconciles the dead-reckoned estimate with the fresh GPS fix smoothly, rather than snapping the position discontinuously.

Prepared as a team-shared explainer — read before pitch rehearsal so everyone can answer any of the above in their own words.